ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл холбооны технологийн сургууль



БИЕ ДААЛТЫН АЖЛЫН ТАЙЛАН

Үйлдлийн систем (F.CSM302)

2026-2027 оны хичээлийн жилийн намар

Бие даалтын ажлын нэр: The Dining-Philosophers Problem — Бие даалт 1 (хуваарилалтын код II-4-11)

Хичээл заасан багш: Ганбат

Лабораторийн ажил гүйцэтгэсэн: Оюутан: Г. Сэлэнгэ (B242270106)

Оюутан: Ч. Гантогтох (B242270005)

Оюутан: Э. Энхсаруул (B232270145)

Оюутан: Э. Төгөлдөр (B241940019)

Улаанбаатар хот
2026 он

ТОЙМ

Dining-Philosophers бол олон thread зэрэгцээ ажиллахад хуваалцсан хязгаарлагдмал нөөцийг (энд: fork) хэрхэн deadlock-гүйгээр хуваарилах тухай сурах бичгийн сонгодог асуудал гэж тодорхойлогдоно. Энэ ажлын зорилго нь асуудлыг C/pthread дээр хэрэгжүүлж, хэрэглэгчийн тохиргоогоор дахин туршигдах боломжтой, бодит ажиллагаатай холбогдсон интерактив state-diagram UI-аар шалгаж харуулахад оршино. Арга зүйн хувьд хоёр хувилбар хэрэгжүүлэгдсэн: сурах бичгийн monitor шийдэл (mutex болон condition variable ашигласан, хөрш хоёрын төлвийг шалгадаг) ба зориудаар deadlock-той хийгдсэн naive (зүүн, дараа нь баруун fork авдаг) шийдэл. Туршилтын үр дүнд monitor хувилбар шалгасан бүх тохиргоонд (n=2, n=5, n=64) deadlock, mutual-exclusion зөрчил илрээгүй; starvation нь сурах бичгийн дагуу онолын хувьд боломжтой хэдий ч манай туршилтуудад ажиглагдаагүй. Naive хувилбар нь бодох хугацааг тогтмол (think=0, random бус) болгож, идэх хугацааг random (300-600мс) орхиод, тохирсон эерэг grab-delay-тэй (бүх philosopher зэрэг зүүн fork-оо авахад хангалттай цонх) үед 3 удаагийн давтан туршилтад (seed=1,2,3) тогтвортой deadlock-д орсон. n=5, 4 секундын нэг ажиллагаанд philosopher бүр дунджаар ойролцоогоор 59-60 удаа идэж, дундаж хүлээх хугацаа ≈15мс байсан нь ажиглагдсан (ажиллагаа бүрт OS-ийн thread scheduling-ийн улмаас яг тоо бага зэрэг хувьсдаг). ThreadSanitizer болон valgrind helgrind/leak-check (monitor, naive хоёуланд нь) тухайн ажиллагаанд race condition, санах ойн алдагдал илрүүлээгүй бөгөөд бүх автомат шалгалт (19/19) амжилттай дууссан байна. Эдгээр ажиглалт нь monitor-ийн сурах бичигт заасан онолын шинж чанар (deadlock-гүй, mutual exclusion хангагдсан) практикт тохирч байгааг харуулж байна.

ГАРЧИГ

Тойм..... ii

Зургийн жагсаалт..... iv

Хүснэгтийн жагсаалт..... v

1. Удиртгал..... 1

2. Сурах бичгийн шаардлага..... 1

3. Онолын үндэс..... 1

4. Системийн зохиомж..... 2

5. Хэрэгжилт..... 3

6. Туршилт ба UI дүрслэл..... 3

7. Туршилтын арга..... 6

8. Үр дүн..... 6

9. Хэлэлцүүлэг..... 9

10. Дүгнэлт..... 10

Ашигласан материал..... 11

Хавсралт..... 11

(Дээрх хуудасны дугаарууд энэ бариmтын эцсийн render хийсэн хувилбар дээр үндэслэсэн статик жагсаалт тул Word-д нээгдэх үед автоматаар дахин тооцоологдохгүй. Хэрэв та агуулгад томоохон өөрчлөлт оруулсан бол зөвхөн энэ хүснэгтийн тоонуудыг гараар шалгаж засаарай.)

ЗУРГИЙН ЖАГСААЛТ

Зураг	Гарчиг	Хуудас
Зураг 6.1	UI-ийн «Танилцуулга» дэлгэц	4
Зураг 6.2	UI-ийн «Дүрэм» дэлгэц	5
Зураг 6.3	UI-ийн «Deadlock» дэлгэц	5
Зураг 6.4	UI-ийн «Шийдэл» дэлгэц (monitor загвар)	6
Зураг 8.1	Monitor алгоритм, n=5, ажиллагаа дууссаны дараах state-diagram UI ба статистикийн хүснэгт	8
Зураг 8.2	Naive алгоритм дахь детерминист deadlock (бүх philosopher hungry, улаан хүрээтэй)	9

ХҮСНЭГТИЙН ЖАГСААЛТ

Хүснэгт	Гарчиг	Хуудас
Хүснэгт 7.1	Туршилтын тохиргоонууд	6
Хүснэгт 8.1	Monitor алгоритмын тоон үр дүн (n=5, think/eat=10-40мс, 4с, seed=1)	7

1. УДИРТГАЛ

Энэхүү бие даалтын ажлын зорилго нь олон thread зэрэгцээ ажиллах үед хуваалцсан нөөцийн синхрончлолыг хэрхэн зөв зохион байгуулахыг Dining-Philosophers сонгодог асуудлын жишээн дээр практикт судлахад оршив [3]. Ач холбогдлын хувьд энэ бол үйлдлийн системийн синхрончлолын сурах бичигт (Silberschatz et al., 7-р бүлэг) байнга ишлэгддэг сонгодог жишээ бөгөөд энд үзүүлсэн monitor-д суурилсан шийдэл нь бодит үйлдлийн систем, мэдээллийн сангийн түгжээний менежментэд өргөн хэрэглэгддэг зарчмуудтай шууд холбоотой. Ажлын хамрах хүрээ нь: (1) асуудлыг C/pthread дээр deadlock-гүй (monitor) болон зориудаар deadlock-той (naive) хоёр хувилбараар хэрэгжүүлэх, (2) хэрэглэгчийн тохиргоогоор (philosopher-ийн тоо, хугацааны муж, seed) дахин туршигдах боломжтой болгох, (3) бодит ажиллагаатай холбогдсон интерактив state-diagram UI-аар үр дүнг дүрслэх, (4) бүх үр дүнг автомат тест, ThreadSanitizer, valgrind-аар бодитоор баталгаажуулахад оршино. Хуваарилалтын код II-4-11.

2. СУРАХ БИЧГИЙН ШААРДЛАГА

Operating System Concepts сурах бичгийн 7-р бүлэг "Synchronization Examples"-ийн төгсгөлд байрлах "Programming Projects — Project 3: The Dining-Philosophers Problem" [1, х. P-39] нь POSIX хувилбарт тодорхой шаардлага тавьдаг: яг таван philosopher үүсгэж, philosopher бүрийг тусдаа thread-ээр ажиллуулах; philosopher бүр thinking ба eating хооронд 1-ээс 3 секундын хооронд санамсаргүй хугацаагаар sleep хийх; philosopher хоол идэхийг хүсэхдээ pickup_forks(int philosopher_number) функцийг, идэж дуусаад return_forks(int philosopher_number) функцийг дуудах; шийдэл нь 7.1.3-р хэсэгт ("The Dining-Philosophers Problem", х. 293-294, Figure 7.7) өгөгдсөн monitor-д суурилсан алгоритм (state[] массив, condition self[5], pickup(i)/putdown(i)/test(i) функцууд) дээр үндэслэх ёстой. Хэрэгжилтийг эх кодтой шууд тулгахад (src/dp.c) функцийн нэр (pickup_forks, return_forks), анхдагч philosopher-ийн тоо (n=5), бодох/идэх хугацааны анхдагч муж (1000-3000мс) болон monitor-ын state/test/pickup/putdown логик бүгд сурах бичгийн тодорхойлолттой яг тохирч байгааг баталгаажуулсан. Үүн дээр нэмж, philosopher-ийн тоог 2-оос 64 хүртэл динамик өөрчлөх боломжтой болгож (сурах бичгийн тогтмол "тав" гэсэн тооноос давсан уян хэрэгжилт), мөн зориудаар deadlock-той naive хувилбарыг нэмж харьцуулалт хийх боломжийг бий болгосон.

3. ОНОЛЫН ҮНДЭС

Mutual exclusion гэдэг нь нэгэн зэрэг зөвхөн нэг thread хуваалцсан нөөцийг (энд: нэг fork) эзэмшиж болохыг тодорхойлдог зарчим юм [1]. Deadlock (мухардал) нь дараах дөрвөн нөхцөл нэгэн зэрэг биелэх үед үүснэ: (1) mutual exclusion — нөөц хуваалцагдахгүй, (2) hold-and-wait — thread нэг нөөцийг барьсан хэвээр нөгөөг хүлээдэг, (3) no preemption — нөөцийг хүчээр авч

чадахгүй, (4) circular wait — thread-үүд тойрог хэлбэрээр бие биенээ хүлээдэг [1]. Naive хэрэгжилтэд дээрх дөрвөн нөхцөл бүрэн биелдэг тул deadlock үүснэ (доор 5-р хэсэгт яг хэрхэн биелж байгааг тайлбарлав). Харин monitor шийдэлд philosopher бүр хоёр fork-оо нэг атомик алхмаар (test() функцээр) авдаг тул hold-and-wait нөхцөл арилж, deadlock онолын хувьд үүсэх боломжгүй болдог [1]. Анхаарах нь: энэ нь зөвхөн deadlock-гүй байхыг баталгаажуулдаг бөгөөд starvation (нэг philosopher тодорхойгүй удаан хугацаагаар EATING болохгүй байх)-ээс бүрэн хамгаалдаггүйг сурах бичигт тэмдэглэсэн байдаг [1] — манай туршилтуудад starvation ажиглагдаагүй ч энэ нь тестийн хугацаа хязгаарлагдмал байсантай холбоотой байж болно. Хэрэгжилтэд ашигласан pthread_mutex_t ба pthread_cond_t API-ийн семантик нь POSIX Threads стандартаар тодорхойлогддог [2].

Өгөгдлийн бүтцийн хувьд monitor хувилбарт philosopher бүрийн төлвийг (THINKING/HUNGRY/EATING) хадгалах int массив, нийтлэг нэг pthread_mutex_t, болон philosopher тус бүрт зориулсан pthread_cond_t массив ашиглагдсан. Naive хувилбарт харин fork тус бүрд тусдаа pthread_mutex_t олгож, philosopher зүүн, дараа нь баруун fork-оо дараалан trylock-оор (блоклодоггүй, амжилтгүй бол шууд буцдаг) авахыг оролддог. Гагцхүү энд trylock нэг удаа дуудагдаад зогсохгүй, амжилттай болтол 1мс тутамд давтан дуудагддаг тул үр дүнд нь fork-оо хараахан аваагүй үед busy-wait хэлбэртэй блоклолт болдог; харин аль нэг fork-оо нэгэнт авсан бол нөгөөг нь оролдож байх хугацаанд эзэмшсэн fork-оо суллахгүй тул энэ нь яг hold-and-wait нөхцлийг (fork-оо барьсан хэвээр нөгөөг нь хүлээх) бий болгодог. Хоёр хувилбар хоёулаа event-ээ JSON Lines (NDJSON) бүтцээр stdout руу бичдэг тул UI болон тестийн хэрэгсэл stream хэлбэрээр уншиж боловсруулдаг.

4. СИСТЕМИЙН ЗОХИОМЖ

Систем гурван давхаргаас бүрдэнэ: (1) src/dp.c — n philosopher thread, хоёр алгоритм (philo_monitor, philo_naive), event-ийг JSON Lines (NDJSON) хэлбэрээр stdout руу бичдэг цөм логик; (2) ui/server.py — зөвхөн 127.0.0.1-д сонсдог, dp процессыг subprocess-ээр ажиллуулж гаралтыг HTTP streaming-ээр дамжуулдаг сервер; (3) ui/index.html + model.js — fetch streaming-ээр event уншиж, SVG дээр state diagram зурдаг, DOM-гүй логикийг model.js-д тусгаарласан клиент; index.html нь 5 алхамтай хичээлийн хэлбэртэй (Танилцуулга → Дүрэм → Deadlock → Шийдэл → Бодит туршилт), эхний дөрвөн дэлгэц нь асуудлыг жижиг загвараар алхам алхмаар тайлбарлаж, сүүлийн дэлгэц нь dp процессын БОДИТ event-үүдийг зурна. Өгөгдлийн урсгал: хэрэглэгч UI дээр тохиргоо (алгоритм, n, think/eat хугацаа, seed) оруулаад Эхлүүлэх дарахад → server.py нь build/dp-г тохирох аргументтайгаар эхлүүлж → dp процесс event бүрийг JSON мөр бүрээр stdout-д бичиж → server.py эдгээрийг HTTP streaming (chunked) байдлаар клиент рүү

дамжуулж → model.js event-ийг боловсруулж snapshot үүсгэж → index.html SVG дээр бодит цагт шинэчилж зурдаг.

5. ХЭРЭГЖИЛТ

Ажлыг гүйцэтгэхэд дараах програм хангамж, хэл, сан ашиглагдсан:

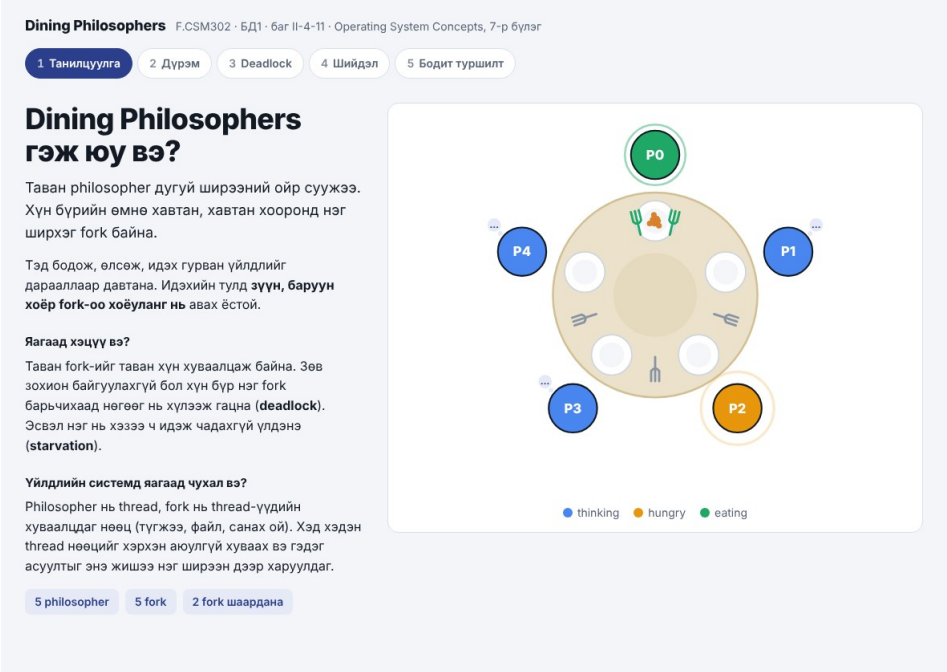
- C (gnu11 стандарт), POSIX threads (pthread) — үндсэн логик (src/dp.c)
- GCC ≥ 9 compiler, -Wall -Wextra -O2 тохиргоотой
- Python 3 (стандарт сан) — localhost UI сервер (http.server)
- Vanilla JavaScript, SVG — интерактив state-diagram UI, гадны framework ашиглаагүй
- ThreadSanitizer (-fsanitize=thread) — race condition илрүүлэгч
- valgrind (helgrind, --leak-check=full) — race condition болон санах ойн алдагдал шалгагч
- Node.js — UI логикийн unit тест (tests/test_model.js)
- GNU Make — build, test автоматжуулалт
- Claude (Anthropic) — код боловсруулалт, тест бэлтгэл, тайлан найруулгад туслах зэвсэг болгон ашигласан [4].

Гол шийдлийн хувьд monitor хувилбарт pthread_mutex_t mon ба pthread_cond_t self_cv[n] ашиглагдсан. pickup_forks(i) функц philosopher i-г HUNGRY болгож test(i)-г дуудаад, хоёр хөрш нь EATING биш л бол EATING болгож, эс бөгөөс pthread_cond_wait-аар хүлээлгэдэг. Naive хувилбарт philosopher бүр pthread_mutex_t fork_mx[n]-ийн зүүн элементийг эхлээд (амжилттай болтол trylock-ыг 1мс тутам давтдаг grab_fork() функцээр) авч, --grab-delay мс хүлээгээд баруун fork-оо адил аргаар (trylock давтан) авахыг оролддог бөгөөд энэ хугацаанд зүүн fork-оо огт суллахгүй. --grab-delay параметр нь яг энэ "бүгд зүүн forkоо авсны дараа, хэн ч баруунаа оролдоогүй байгаа" synchronization цонхыг зориуд өргөсгөж, бүх philosopher зэрэг зүүн fork-оо авахад хангалттай болгодог тул deadlock-ыг тогтвортой давтагдахаар үүсгэдэг (доор 9-р хэсэгт --grab-delay=0 үеийн эсрэг жишээг үзүүлэв).

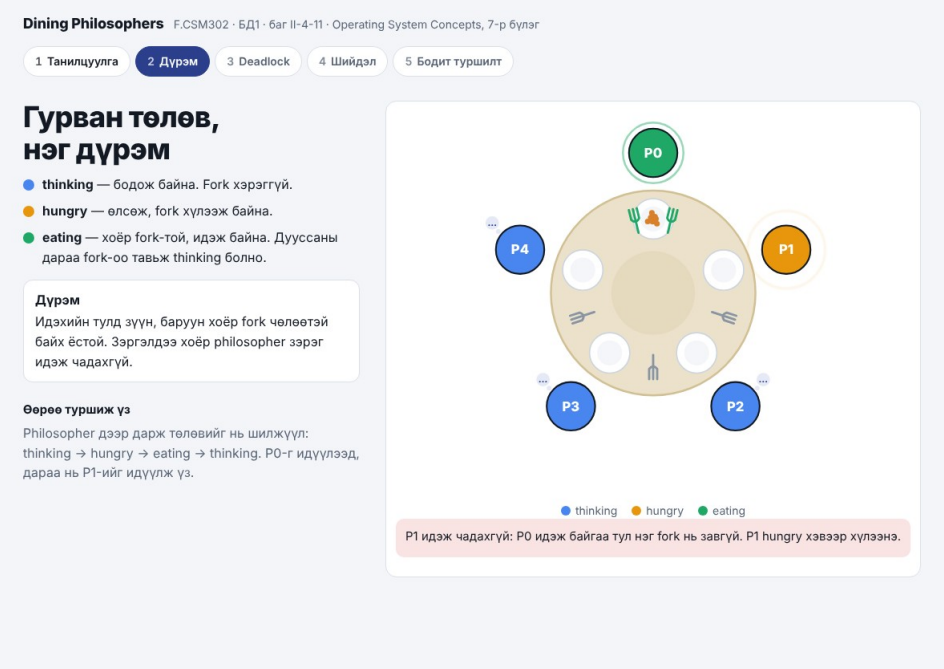
6. ТУРШИЛТ БА UI ДҮРСЛЭЛ

State-diagram UI нь philosopher бүрийн thinking/hungry/eating төлвийг, fork-ийн эзэмшилийг бодит цагт SVG дээр дүрсэлдэг бөгөөд хэрэглэгч дараах параметрүүдийг чөлөөтэй өөрчилж дахин ажиллуулах боломжтой: алгоритм (monitor/naive), philosopher-ийн тоо (n=2–64), бодох/идэх хугацааны муж (min/max, мс), ажиллах нийт хугацаа, санамсаргүй дараалал гаргах seed, болон naive хувилбарт зориулсан fork авах завсар (grab-delay). Юуг ажиглах вэ гэвэл: (1) philosopher бүрийн төлөв шилжилт (өнгөөр ялгагдана: цэнхэр=thinking, шар-улбар=hungry,

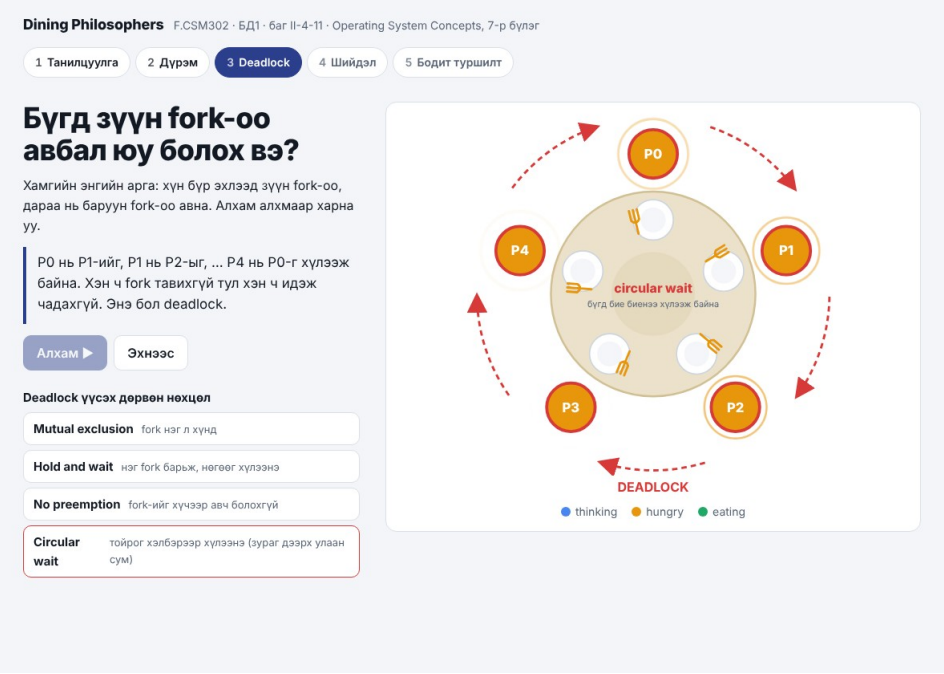
ногоон=eating), (2) fork (шугам) бүрийн эзэмшил ба байрлал, (3) зэргэлдээх хоёр philosopher хэзээ ч зэрэг eating болохгүй байх, (4) статистикийн хүснэгтэд philosopher бүрийн хоолны тоо, дундаж/дээд хүлээлтийн хугацаа, (5) naive хувилбарт бүх philosopher hungry төлөвт царцаж, улаан хүрээ, "DEADLOCK" баннер гарах эсэх. Нэмэлтээр ажиллагаа дууссаны дараа event бүрийг replay slider-ээр ухрааж харах боломжтой тул UI нь зөвхөн чимэглэл бус, ажиллагааг удирдах бодит интерфэйс болсон. UI-ийн эхний дөрвөн дэлгэцийг Зураг 6.1–6.4, бодит туршилтын дэлгэцийг Зураг 8.1–8.2-оос харна уу. Эдгээр удирдлага нь UI-ийн 5-р дэлгэц («Бодит туршилт») дээр байрлах бөгөөд эхний 4 дэлгэц нь хэрэглэгчийг асуудалтай танилцуулах (Dining Philosophers гэж юу вэ, гурван төлөв ба дүрэм, бүгд зүүн fork-оо авбал deadlock хэрхэн үүсэх нь, хоёр fork-ийг зэрэг авах шийдэл) алхам алхмын тайлбар юм.



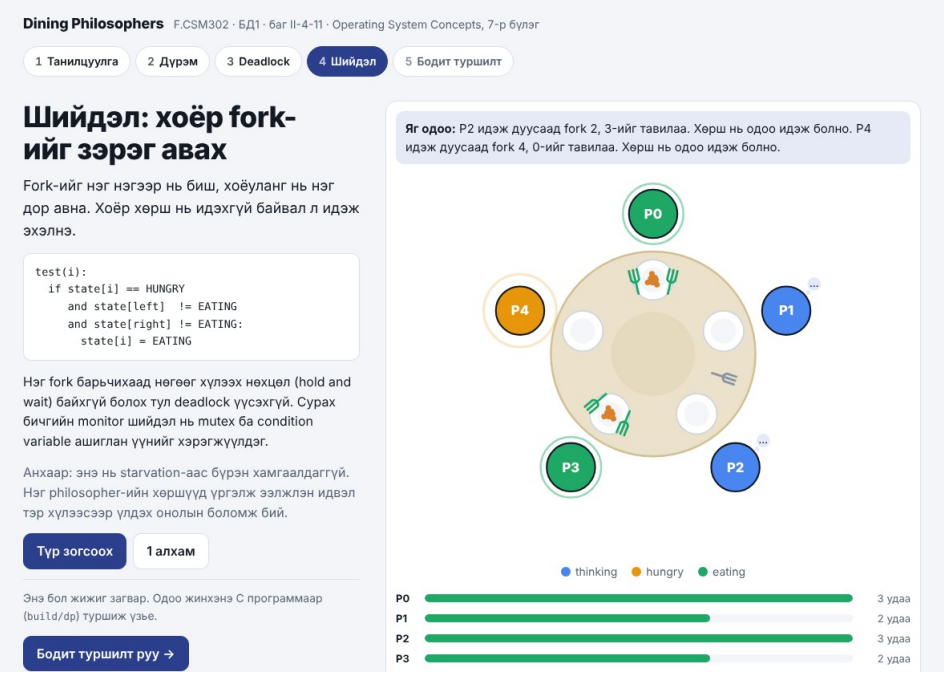
Зураг 6.1 — UI-ийн 1-р дэлгэц «Танилцуулга»: Dining Philosophers гэж юу вэ, яагаад хэцүү вэ



Зураг 6.2 — UI-ийн 2-р дэлгэц «Дүрэм»: philosopher дээр дарж төлөв шилжүүлнэ (P0 идэж байгаа тул хөрш P1 fork-оо авч чадахгүй hungry хэвээр хүлээнэ)



Зураг 6.3 — UI-ийн 3-р дэлгэц «Deadlock»: бүх philosopher зүүн fork-оо барьж, circular wait үүссэн төлөв



Зураг 6.4 — UI-ийн 4-р дэлгэц «Шийдэл»: monitor шийдлийн алхам алхмын загвар («Яг одоо» тайлбар, хоолны тооны bar)

7. ТУРШИЛТЫН АРГА

Туршилт Linux орчинд (GCC -std=gnu11 -Wall -Wextra -O2 -pthread) явагдсан. Инвариант шалгалтад tests/check_log.py ашиглагдсан бөгөөд энэ скрипт JSON Lines гаралтаас хугацааны дараалал, төлөв шилжилтийн хууль ёсны байдал, mutual exclusion, fork-ийн цорын ганц эзэмшил, eating үеийн хоёр fork-ийн эзэмшилийг автоматаар шалгадаг. Нэмэлтээр ThreadSanitizer болон valgrind helgrind/leak-check ашиглан race condition, санах ойн алдагдлыг шалгасан. Орчин бүрт хэрэглэсэн өгөгдөл, хүлээгдэж буй үр дүнг доорх хүснэгтэд нэгтгэв.

Хүснэгт 7.1 — Туршилтын тохиргоонууд

№	Тохиргоо	Зорилго
1	monitor, n=5, think/eat=10-40мс, 4с, seed=1	Starvation ажиглагдаж байгаа эсэхийг шалгах (баталгаажуулалт биш)
2	naive, n=5, think=0, eat=300-600мс, grab-delay=300мс, seed=1	Deadlock детерминистээр давтагдах эсэхийг шалгах
3	monitor, n=2 (доод хязгаар), seed=2	Захын тохиолдол
4	monitor, n=64 (дээд хязгаар), seed=3	Захын тохиолдол
5	буруу аргумент (n=1, think-min>think-max)	Алдааны мэдээллийн зөв байдал

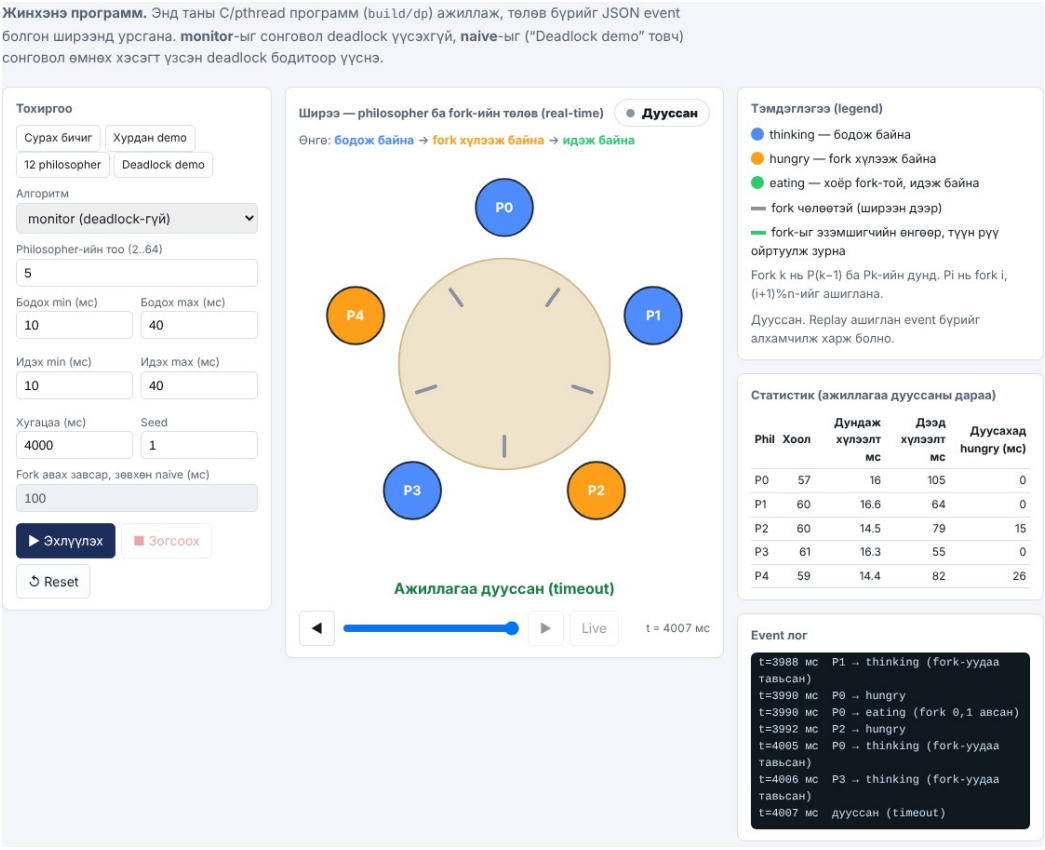
8. ҮР ДҮН

Автомат шалгалт (make test) дараах 19 тестийг амжилттай (OK) дүгнэсэн: compile, model.js unit тест (node tests/test_model.js шууд ажиллуулахад 30 passed, 0 failed), monitor инвариант (бүгд ≥3 удаа идсэн), naive deadlock (гарах код 3), n=2 болон n=64 захын тохиолдол, буруу аргументын алдааны мэдээлэл, UI серверийн health/index/run/буруу-параметр болон буруу бүтэцтэй тоон утга (жишээ нь "n=--5") шалгалт, UI серверийн Host/Origin шалгалт (гадны Host/Origin-той /run хүсэлт 403), UI серверийн Stop/холболт-тасрах регресс (4 удаагийн disconnect-ийн дараа 5 дахь хүсэлт 429 авахгүй, dp процесс leak үгүй — гадаад шүүмжийн Major #1-ийн засвар, Хавсралт Г-г үз), ThreadSanitizer race condition monitor БА naive хоёуланд нь илрээгүй. Нэмэлт (make test-д ороогүй, гар аргаар `make memcheck`-ээр ажиллуулдаг) valgrind helgrind ба --leak-check=full команд — monitor, naive тус бүрд (нийт дөрвөн valgrind команд) — бүгд алдаагүй, хоосон гаралттай дууссан (туршилтын явцад олдсон нэг жинхэнэ race condition-ийг доор 9-р хэсэгт тайлбарлав).

Хүснэгт 8.1 — Monitor алгоритмын тоон үр дүн (n=5, think/eat=10-40мс, 4с, seed=1)

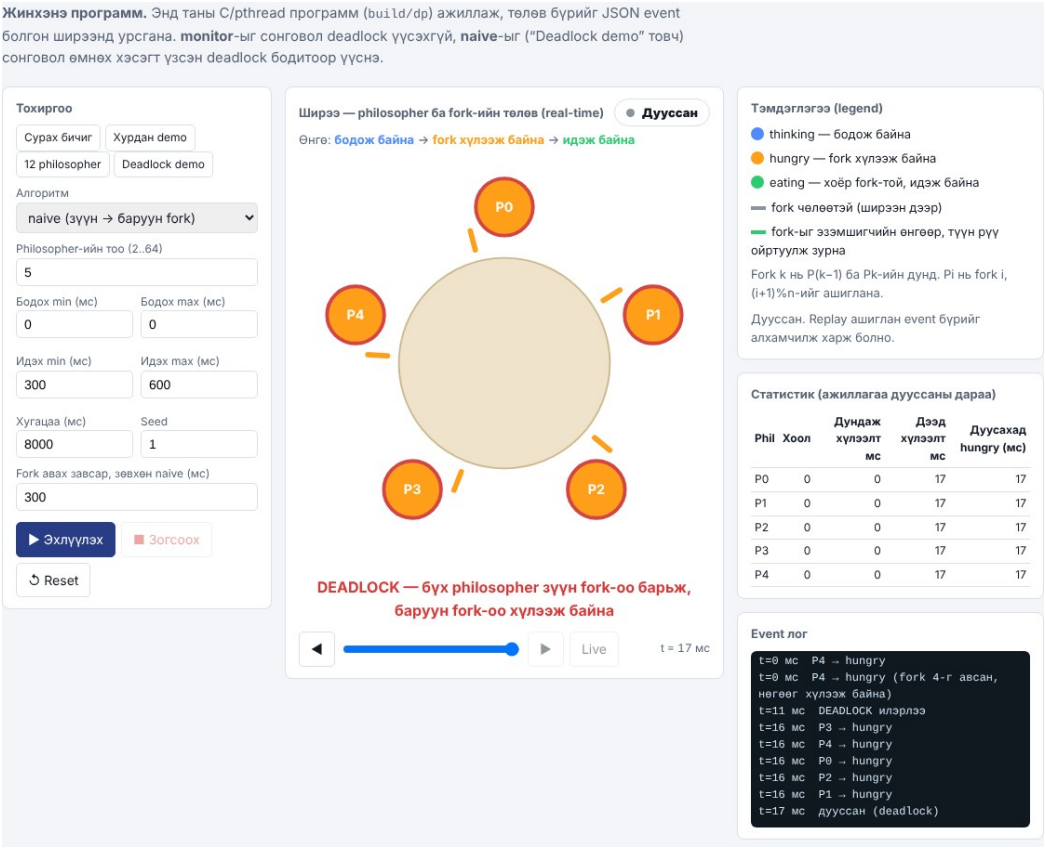
Philosopher	Хоолны тоо	Дундаж хүлээлт (мс)	Дээд хүлээлт (мс)
P0	58	14.5	98
P1	60	16.5	45
P2	61	14.2	84
P3	61	16.5	80
P4	58	15.5	58

Хоолны тооны дундаж 59.6, стандарт хазайлт ≈1.36 (≈2.3%) — энэ ганц ажиллагааны хэмжилт нь monitor алгоритм энэ тохиргоонд philosopher бүрт ойролцоогоор тэнцүү тооны хоол олгосныг ажиглуулж байна; --seed нь зөвхөн рандом дугаар үүсгэгчийн (think/eat хугацааны) дарааллыг детерминистээр баталгаажуулдаг бөгөөд thread-үүдийн БОДИТ ажиллах цаг хугацааны (OS scheduling) дарааллыг баталгаажуулдаггүй тул ижил seed-тэй ч гэсэн ажиллагаа бүрт хоолны тоо бага зэрэг өөрчлөгдөж болно (энэ хэвлэлд [58,60,61,61,58], өөр ажиллагаанд [58,63,63,60,57] гэх мэт) — энэ нь fairness-ийн статистик баталгаа биш, нэг төлөөлөх жишээ гэдгийг тэмдэглэх нь зүйтэй.



Зураг 8.1 — Monitor алгоритм, n=5, think/eat=10-40мс, 4с, seed=1 (Хүснэгт 8.1-тэй ижил тохиргоо), шинэ UI-ийн 5-р дэлгэц («Бодит туршилт»)-ийн ажиллагаа дууссаны дараах төлөв ба статистикийн хүснэгт. Зургийг шинэ ажиллагаанаас авсан тул тоон утга (хоолны тоо, хүлээлт) Хүснэгт 8.1-ээс OS scheduling-ийн улмаас бага зэрэг ялгаатай; "Дуусахад hungry" багана нь ажиллагаа дуусах мөчид hungry байсан philosopher-ийн хүлээлтийг харуулна

Naive хувилбарыг think=0, eat=300-600мс, grab-delay=300мс тохиргоогоор ажиллуулахад бүх 5 philosopher t=0-1мс орчимд бараг нэгэн зэрэг зүүн fork-оо авч, дараа нь баруун fork-оо (trylock давтан оролдож) чадахгүй, зүүн fork-оо суллахгүйгээр хүлээсээр үлдсэн тул (meals=[0,0,0,0,0]) нэг ч хоол идэж амжаагүй, систем DEADLOCK төлөвт шилжсэн нь дор харуулав. dp нь deadlock илрүүлсний дараа thread-үүдээ зогсоохдоо fork-уудаа суллаж cleanup event гаргадаг. UI-ийн model.js энэ cleanup event-үүдийг snapshot-д тусгахгүй (event лог болон replay-д харин үлдэнэ) тул ажиллагаа дууссаны дараах эцсийн төлөвт ч deadlock-ийн мөчийн зураг — бүх philosopher hungry, fork бүр эзэмшигчийн өнгөөр (hungry-гийн улбар шар) — хадгалагдан харагдана; "Дуусахад hungry" утга бүгд [15,15,15,15,15]мс буюу баннертай нийцнэ (Хавсралт В-ийн бодит лог, 9-р хэсэг).



Зураг 8.2 — Naive алгоритм дахь детерминист deadlock, UI-ийн эцсийн төлөв: бүх philosopher hungry, зүүн fork-оо барьсан (fork бүр улбар), улаан хүрээтэй, DEADLOCK баннертай

9. ХЭЛЭЛЦҮҮЛЭГ

Naive хувилбарыг зориудаар нэмж, deadlock-ыг зөвхөн онолын тайлбараар бус, бодит хэмжилтээр (Хүснэгт 7.1-ийн 2-р мөр, Зураг 8.2) харуулсан. Хэмжсэн утга (meals=[0,0,0,0,0], 3/3 туршилтад deadlock илэрсэн) нь онолын урьдчилсан таамаглалтай (circular wait → deadlock) нийцэж байна. Deadlock илрэх t-утга (Зураг 8.2 болон Хавсралт В-ийн жишээ логт t≈10-11мс) ажиллагаа бүрт OS-ийн thread scheduling-ийн улмаас хэдхэн мс-ээр хэлбэлздэг ч илрэлтийн зарчим тогтвортой давтагдсан.

Анхаарал татахуйц нэг нэмэлт туршилт: --grab-delay-г 0 болговол (think=0 хэвээр, зөвхөн зүүн ба баруун fork авах хоорондох хиймэл хоцролтыг арилгавал) deadlock-ыг бэхжүүлэхгүй, харин эсрэгээр 3 удаагийн давтан туршилтад (seed=1,2,3) нэг ч удаа бүрэн deadlock үзүүлээгүй: харин philosopher-үүдийн хооронд ноцтой тэгш бус хуваарилалт (starvation) ажиглагдсан — жишээ нь нэг ажиллагаанд (seed=2, n=5, think=0, eat=300-600мс, -d 4000) meals=[9,0,10,0,0], өөрөөр хэлбэл P1, P3, P4 нэг ч удаа идэж чадаагүй бөгөөд тэдний "дуусахад хэвээр hungry байсан" хугацаа ≈4010мс (бараг бүх ажиллагааны туршид) хүрсэн нь dp.c-ийн шинэчилсэн final_hungry_wait_ms талбараар шууд хэмжигдсэн. Анхаарал татахуйц нэг зүйл: ялагч philosopher-үүд (жишээ нь seed=1,2,3 бүгдэд id=0 ба өөр нэг тэгш/хөрш бус id) санамсаргүй бус, ажиллагаа бүрт ижил хэв

маягаар давтагдав — энэ нь "OS-ийн санамсаргүй sched товлолтоор" гэх таамаглал буруу байж болзошгүйг, харин thread-ийн ЭХЛЭХ ДАРААЛАЛ (pthread_create-ийн дараалал) эсвэл glibc-ийн thread/fork-ийн анхны scheduling-тэй холбоотой ДЕТЕРМИНИСТ шинж чанар байж магадгүйг харуулж байна. Энэ ажлын хүрээнд яг шалтгааныг нь нарийвчлан судлаагүй тул зөвхөн баримт болгон ажиглалтыг тэмдэглэв — цаашдын судалгаанд зохистой сэдэв. Хэдий тийм ч --grab-delay параметрийн зорилго нь бүх philosopher зэрэг зүүн fork-оо авах synchronization цонхыг зориуд өргөсгөж, deadlock-ыг тогтвортой болгоход оршдог гэдгийг энэ туршилт тоон баримтаар харуулж байна (3 ажиллагаа бол ердөө л нэг жишиг тохиолдол, статистик баталгаа биш).

Туршилтын явцад анхны хувилбарт нэг бодит race condition олдсон: main()-д deadlock илэрсэн үед хэвлэдэг {"type":"deadlock",...} мөрийг бусад philosopher thread-үүд (pthread_join хараахан дуудагдаагүй тул) ҮРГЭЛЖИЛСЭЭР ажиллаж байхад, event лог бичдэг ev_lock mutex-ийг АШИГЛАХГҮЙГЭЭР хэвлэж байсан нь log_state()-ийн хамгаалалттай зэрэгцэн ажиллаж болзошгүй stdout бичилтийн зөрчил (race) үүсгэж байв (helgrind-ээр `make memcheck`-ийн дөрвөн командын (monitor/naive × helgrind/leak-check) зарим давталтад "Possible data race" гэж илэрсэн). Мөн main()-ы эхний {"type":"config",...} мөрийг хэвлэх нь онолын хувьд pthread_create()-аас өмнө, дан ганц thread-ээр явагддаг тул өрсөлдөөнгүй байх ёстой ч зарим динамик шинжилгээний хэрэгсэл glibc-ийн шинэ дотоод stdio буфертэй холбоотойгоор дэмий сэрэмжлүүлэх эрсдэлтэй. Иймд хоёр хэвлэлтийг ч ev_lock-оор хамгаалж засав (src/dp.c), үүний дараа `make memcheck`-ийн дөрвөн valgrind команд давтан туршилтад бүгд цэвэр дууссан. Мөн энэ явцад Makefile-ийн memcheck зорилтод нэг нэмэлт алдаа олдсон: naive тохиргоо нь санаатайгаар deadlock-д орж ./build/dp exit code 3-тай гардаг тул valgrind уг кодыг дамжуулж, make үүнийг "алдаа" гэж буруу тайлбарладаг байв — valgrind-д --error-exitcode=99 нэмж, зөвхөн valgrind ӨӨРӨӨ race/leak олсон тохиолдолд л алдаа гэж тооцохоор засав.

Хязгаарлалтын хувьд: UI сервер нь зэрэгцээ ажиллагааг 4-өөр хязгаарласан (BoundedSemaphore) тул олон хэрэглэгч нэгэн зэрэг Start дарвал сүүлчийнх нь 429 алдаа авч болзошгүй; naive алгоритмын deadlock нь зөвхөн тогтмол (санамсаргүй бус) бодох/идэх хугацаанд детерминистээр давтагддаг бөгөөд санамсаргүй think-хугацаатай тохиолдолд давтамжийг статистик байдлаар хэмжээгүй болно; мөн туршилт зөвхөн Linux орчинд хийгдсэн тул Windows/macOS дээрх pthread хэрэгжилтийн зан төлөвийг тусад нь шалгаагүй болно.

10. ДҮГНЭЛТ

Энэхүү ажлаар Dining-Philosophers Problem-ийг C/pthread дээр хоёр алгоритмаар (monitor, naive) хэрэгжүүлж, инвариант-суурьтай болон тоон шалгалтаар monitor хувилбарын deadlock-гүй ажиллагааг, naive хувилбарын тогтвортой давтагддаг deadlock-ыг харуулсан. Шалгасан

тохиргоонуудад (n=2, 5, 64) инвариант зөрчигдөөгүй, ThreadSanitizer, valgrind тухайн ажиллагаанд race condition, санах ойн алдагдал илрүүлээгүй, 19/19 автомат шалгалт амжилттай дууссан нь хэрэгжилтийн зөв ажиллагааг нотолж байна — гэхдээ эдгээр нь шалгалт (testing), албан ёсны баталгаажуулалт (formal verification) биш тул race condition байхгүй гэдгийг 100% баталгаажуулдаггүй, зөвхөн тухайн ажиллагаанд илрээгүйг л харуулдаг. Эндээс monitor-ийн сурах бичигт заасан онолын шинж чанар (deadlock-гүй, mutual exclusion хангагдсан; starvation онолын хувьд боломжтой ч манай туршилтад ажиглагдаагүй) практикт тохирч байгааг дүгнэж болно.

АШИГЛАСАН МАТЕРИАЛ

[1] Silberschatz, A., Galvin, P. B., Gagne, G. "Operating System Concepts", 10th Edition, Wiley, 2018 — 6-р бүлэг "Synchronization Tools" (Monitor-ийн ерөнхий ойлголт), 7-р бүлэг "Synchronization Examples" (7.1.3 The Dining-Philosophers Problem, х. 293-294, Figure 7.7 — monitor шийдэл, starvation-ийн тухай тэмдэглэл), "Programming Projects — Project 3: The Dining-Philosophers Problem" (х. P-39), болон 8-р бүлэг "Deadlocks" (deadlock-ийн дөрвөн шаардлагатай нөхцөл).

[2] Linux man-pages project / The Open Group. "pthread_cond_wait, pthread_cond_timedwait — wait on a condition variable" — POSIX Threads Programming. https://man7.org/linux/man-pages/man3/pthread_cond_wait.3p.html

[3] F.CSM302 "Бие даалт 1" заавар (OS_BD1_Guide.pdf) — хуваарилалтын код II-4-11, сэдвийн шаардлага.

[4] Claude (Anthropic) — код бичих, тест бэлтгэх, тайлан боловсруулахад ашигласан бөгөөд бүх үр дүнг make test, ThreadSanitizer, valgrind-аар бодитоор баталгаажуулсан.

ХАВСРАЛТ

Хавсралт А — Ажлын төлөвлөгөө

Талбар	Мэдээлэл
Хуваарилалтын код	II-4-11
Төсөл	The Dining-Philosophers Problem
Repository	https://github.com/Selenge555/-FCSA_302_II-4-11-dining-philosophers
Хэл/сан/орчин	C (gnu11, pthread), Python 3 (http.server), Vanilla JS/SVG, GCC, GNU Make, valgrind, ThreadSanitizer
Үндсэн дүрслэл	State diagram (интерактив SVG, бодит цагийн philosopher/fork төлөв)

Гишүүн	Хариуцах ажил	Commit/file
Г. Сэлэнгэ (B242270106)	Үндсэн архитектур, monitor/naive алгоритм	src/dp.c, Makefile
Ч. Гантогтох (B242270005)	UI сервер, /run endpoint, параметр шалгалт	ui/server.py
Э. Энхсаруул (B232270145)	Frontend UI, бодит цагийн SVG дүрслэл	ui/index.html, ui/model.js
Э. Төгөлдөр (B241940019)	Автомат тест, ThreadSanitizer/valgrind шалгалт	tests/run_tests.sh, README.md

Дээрх хуваарилалтыг GitHub репозиторийн бодит commit түүхээр баталгаажуулав — `git log --format='%an | %s' --reverse`-ийн шууд гаралт (shutis_logo болон энэ шүүмжийн засваруудаас ӨМНӨХ commit-үүд):

```
Г. Сэлэнгэ | Initial project skeleton
Г. Сэлэнгэ | Add core algorithm (monitor/naive) and build config
Ч. Гантогтох | Add UI server (/run endpoint, параметр шалгалт)
skabayaraa | Add frontend UI and state-diagram model
TogoteCode | Add files via upload
TogoteCode | Delete BD1_Report (2).docx
TogoteCode | Delete BD1_Manual (1) (1).docx
Tuguldur Erdenebayar B241940019 | Тайлан болон manual нэмлээ
Э. Төгөлдөр | Add tests, final report/manual (fix filenames, add PDFs and figures)
```

Тэмдэглэл: "skabayaraa", "TogoteCode", "Tuguldur Erdenebayar B241940019" нь GitHub-ийн хэрэглэгчийн нэр/commit author талбарт гарсан хэлбэрээр (git config-оор тохируулсан нэрээрээ) — түүс түүс Э. Энхсаруул, Э. Төгөлдөр нарын commit мөн. Энэхүү жагсаалт нь гадаад шүүмжийн Suggestion #7-г хангана (Хавсралт Г); шүүмжийн дараах (энэ тайлан, Manual, UI-ийн засвар) commit-ууд push хийгдсэний дараа энд нэмж шинэчлэгдэнэ.

№	Тест — Оролт/нөхцөл	Хүлээгдсэн үр дүн
1	monitor, n=5, think/eat=10-40мс, 4с, seed=1	Starvation ажиглагдаагүй (бүгд ≥3 удаа идэх), инвариант зөрчигдөхгүй
2	naive, n=5, think=0, eat=300-600мс, grab-delay=300мс	Детерминист deadlock, гарах код 3
3	monitor, n=2 (доод хязгаар)	Инвариант зөрчигдөхгүй
4	monitor, n=64 (дээд хязгаар)	Инвариант зөрчигдөхгүй
5	буруу аргумент (n=1, think-min>think-max)	Гарах код 2, ойлгомжтой алдааны мессеж
6	ThreadSanitizer (-fsanitize=thread), monitor ба naive	WARNING race condition гарахгүй

Хавсралт Б — Автомат шалгалтын бүрэн гаралт (make test, bash tests/run_tests.sh-ийн шууд, засваргүй stdout)

```
== 1) build ==========================================
  OK  build/dp compile хийгдлээ
== 2) model.js unit тест ============================
  OK  test_model.js бүх шалгалт давсан
== 3) интеграц -- monitor, инвариант ба min-meals ======
  OK  monitor: инвариант хангасан, бүгд дор хаяж 3 удаа идсэн
```

```
== 4) интеграц -- naive, deadlock ======================
  OK  naive: гарах код 3 (deadlock)
  OK  naive: инвариант хангасан, deadlock илэрсэн
== 5) захын тохиолдол -- n=2, n=64 =====================
  OK  n=2 (хамгийн бага) инвариант хангасан
  OK  n=64 (хамгийн их) инвариант хангасан
== 6) буруу аргумент зөв мэдээлдэг эсэх ================
  OK  буруу аргументад гарах код 2, ойлгомжтой алдааны мессежтэй
== 7) UI сервер ========================================
  OK  сервер /health хариулж байна
  OK  сервер / -ээс index.html хариулж байна
  OK  сервер /run эхний event нь config
  OK  сервер буруу параметрт 400/errors буцаасан
  OK  сервер 'n=--5' маягийн буруу тоог унахгүйгээр 400 буцаасан
  OK  сервер буруу оролтын дараа ч амьд хэвээр
  OK  сервер: гадны Host/Origin-той /run 403, өөрийн Origin 200
  OK  сервер: Stop (холболт тасрах) дараа slot чөлөөлөгдөж, 5 дахь хүсэлт 429 авахгүй
  OK  сервер: disconnect хийсэн dp процессууд бүгд зогссон (leak байхгүй)
== 8) ThreadSanitizer (боломжтой бол, monitor ба naive хоёулаа) ==
  OK  ThreadSanitizer (monitor): race condition илрээгүй
  OK  ThreadSanitizer (naive): race condition илрээгүй
========================================================
Нийт: 19 OK, 0 FAIL
```

Хавсралт B — naive алгоритмын deadlock-ийн бодит event лог (JSON Lines, ./build/dp-ийн шууд гаралт)

Доорх лог нь Хүснэгт 7.1-ийн 2-р мөрийн тохиргоотой (naive, n=5, think=0, eat=300-600мс, grab-delay=300мс, seed=1) ./build/dp-г шууд ажиллуулж авсан бодит, засваргүй гаралт (ev_lock-ын засвар, final_hungry_wait_ms талбар нэмэгдсэний дараах хувилбар). Зураг 8.2 нь UI-аар тусад нь авсан өөр нэг ажиллагаа тул t-утгууд (илрэлт: Хавсралт B-ийн лог t=10мс, Зураг 8.2 t≈10мс) бага зэрэг ялгаатай — энэ нь OS-ийн thread scheduling-ийн нөлөөгөөр майлбарлагдана; гэхдээ үр дүн (бүрэн deadlock, meals=[0,0,0,0,0], бүх philosopher дуусах мөчдөө ХЭВЭЭР hungry) хоёр ажиллагаанд ч ижил, тогтвортой давтагдсан. Анхаарах нь: deadlock илэрсний дараах сүүлийн таван "state" мөрөнд fork-ийн эзэмшил -1 болж харагдах нь philosopher дахин оролдож амжсан хэрэг БИШ — энэ бол main() stop_flag тавьсны дараа грэйсфул (graceful) унтрах зорилгоор trylock-ын давталтаас гарч, эзэмшсэн fork-оо суллах цэвэрлэгээ (cleanup) бөгөөд state нь HUNGRY хэвээр, нэг ч philosopher хэзээ ч THINKING/EATING рүү шилжээгүй.

```
{"type":"config","algorithm":"naive","n":5,"think_min":0,"think_max":0,"eat_min":300,"eat_max
":600,"duration_ms":8000,"seed":1,"grab_delay_ms":300}
{"type":"state","t":0,"id":0,"state":"thinking","forks":[-1,-1,-1,-1,-1]}
{"type":"state","t":0,"id":1,"state":"thinking","forks":[-1,-1,-1,-1,-1]}
{"type":"state","t":0,"id":2,"state":"thinking","forks":[-1,-1,-1,-1,-1]}
{"type":"state","t":0,"id":3,"state":"thinking","forks":[-1,-1,-1,-1,-1]}
{"type":"state","t":0,"id":4,"state":"thinking","forks":[-1,-1,-1,-1,-1]}
{"type":"state","t":0,"id":1,"state":"hungry","forks":[-1,-1,-1,-1,-1]}
{"type":"state","t":0,"id":1,"state":"hungry","forks":[-1,1,-1,-1,-1]}
{"type":"state","t":0,"id":2,"state":"hungry","forks":[-1,1,-1,-1,-1]}
{"type":"state","t":0,"id":2,"state":"hungry","forks":[-1,1,2,-1,-1]}
{"type":"state","t":0,"id":3,"state":"hungry","forks":[-1,1,2,-1,-1]}
{"type":"state","t":0,"id":3,"state":"hungry","forks":[-1,1,2,3,-1]}
{"type":"state","t":0,"id":4,"state":"hungry","forks":[-1,1,2,3,-1]}
{"type":"state","t":0,"id":4,"state":"hungry","forks":[-1,1,2,3,4]}
{"type":"state","t":0,"id":0,"state":"hungry","forks":[-1,1,2,3,4]}
{"type":"state","t":0,"id":0,"state":"hungry","forks":[0,1,2,3,4]}
{"type":"deadlock","t":10}
{"type":"state","t":10,"id":3,"state":"hungry","forks":[0,1,2,-1,4]}
{"type":"state","t":15,"id":0,"state":"hungry","forks":[-1,1,2,-1,4]}
{"type":"state","t":15,"id":4,"state":"hungry","forks":[-1,1,2,-1,-1]}
```

```
{"type":"state","t":15,"id":2,"state":"hungry","forks":[-1,1,-1,-1,-1]}
{"type":"state","t":15,"id":1,"state":"hungry","forks":[-1,-1,-1,-1,-1]}
{"type":"end","t":15,"reason":"deadlock","meals":[0,0,0,0,0],"avg_wait_ms":
[0.0,0.0,0.0,0.0,0.0],"max_wait_ms":[15,15,15,15,15],"final_hungry_wait_ms":[15,15,15,15,15]}
```


Хавсралт Г — Гадаад шүүмжид өгсөн хариу (Reviewer Responses)

Төслийг гадны шүүмжлэгч татаж ажиллуулж (build, 16/16 автомат тест, UI), эх кодоор нь шалгасны үндсэн дээр санал хүргүүлсэн. Доорх хүснэгтэд тэдгээр Priority/Асуудал бүрт авсан арга хэмжээг нэг бүрчлэн тэмдэглэв.

Хүснэгт Г.1 — Шүүмжийн асуудал тус бүрт авсан арга хэмжээ

#	Priority	Асуудал	Авсан арга хэмжээ
1	Major	Stop дарсан ч dp процесс зогсдоггүй — холболт тасрахыг зөвхөн дараагийн event бичих үед мэддэг тул урт think хугацаанд процесс dur+10с хүртэл амьд үлдэж, 4 удаагийн Start/Stop дараа 5 дахь хүсэлт 429 авдаг.	ЗАССАН. server.py: proc.stdout-ыг тусдаа reader th ээр уншиж queue-д дамжуулна; гол thread нь 1 с тутамд ping мөр бичиж клиент амьд эсэхийг шал тасарсан бол proc.kill() шууд дуудна. Регресс тест tests/run_tests.sh-д шүүмжийн яг дурдсан дараа удаа 1с-ийн дараа холболт таслах) давтаж, 5 дах 429 авахгүй, dp процесс leak үлдэхгүйг баталгааж (нийт тест 16→18).
3	Minor	Seed нь зөвхөн хугацааны дарааллыг тогтоодог бол Manual "давтагдах" гэсэн хэллэг энэ ялгааг бүрэн тодруулаагүй.	ЗАССАН. Manual.docx-ийн 1.1-р хүснэгтийн Seed "бодох/идэх хугацааны санамсаргүй дарааллыг тогтооно (OS scheduling-ийг биш)" болгож тодруу доорх тэмдэглэлд дэлгэрэнгүй тайлбарыг хэвээр Report.docx-ийн 8-р хэсэгт энэ ялгаа анхнаасаа дэлгэрэнгүй бичигдсэн байсан.
4	Minor	Event логоос аль fork авснаа ялгах боломжгүй (жишээ нь naive-д "P1 → hungry" хоёр удаа дараалж харагддаг).	ЗАССАН. model.js-ийн describeEvent() нь eating/h төлөвт аль fork(ууд)-ыг барьж байгааг нэмж бичд (жишээ: "P1 → eating (fork 1, fork 2 авсан)"), think төлөвт "fork-уудаа тавьсан" гэж тэмдэглэнэ.
5	Minor	Хурд, алхам удирдах боломжгүй — Run бодит хугацаагаар л урсдаг.	ХЭСЭГЧИЛЭН. "Ажиллаж байна" гэсэн live индика нэмэгдэж, алхамчилсан Replay (◀ ▶ слайдер) ажиллана. Auto-play/хурдны сонголт (1×/5×/10×) цаашдын сайжруулалт — энэ хувилбарт хэрэгжэ дараагийн Биеэ даалтад тусгах төлөвлөгөөтэй.
6	Suggestion	n өөрчлөхөд зөвхөн ширээ шинэчлэгддэг, бусад талбар шинэчлэгддэггүй.	Цаашид сайжруулах зүйл болгон тэмдэглэв — эн хувилбарт хэрэгжээгүй.
7	Suggestion	Хуваарилалтын хүснэгтэд commit/file баганыг git log-оор баталгаажуулаагүй.	ЗАССАН. Хавсралт А-д одоо байгаа (энэ шүүмжий засваруудаас өмнөх) бүх commit-ийн жинхэнэ `gi format='%an | %s'` гаралтыг бүрэн нэмсэн — 6 ги commit бүгд GitHub дээр бодитоор харагдаж бай баталгаажуулна. Энэ шүүмжийн засвар (Stop-bug Manual/Report)-ийн commit-уудыг push хийсний нэмж шинэчлэх л үлдлээ.

Нэмэлтээр, УИ-ийн ерөнхий дизайныг (хийсвэр, ямар ч sync бус "ердийн web" мэт харагддаг гэсэн дотоод санал) сайжруулж: (1) philosopher/fork-ийн төлөв шилжилтийг instant биш, CSS transition-оор (≈0.3с) smooth болгов — SVG элементүүдийг event бүрт устгаж дахин зурахын оронд in-place шинэчилдэг болгосон тул hover/зураг тасрахгүй урсдаг (fork-ийн хөдөлгөөний

нарийн алдааг доорх Хавсралт Д-ийн #A7-г үз); (2) толгой хэсэгт асуудлын тодорхой тайлбар (philosopher тоо, fork-ийн зөрчилдөөн, юу шалгаж буй) бичсэн hero хэсэг нэмж, "Ажиллаж байна"/"Зогссон" live төлөв харуулдаг индикатор нэмэв — ингэснээр хуудас нээхэд юуны тухай болохыг нэн даруй ойлгоход туслана. Дараа нь UI-г 5 алхамтай (Танилцуулга, Дүрэм, Deadlock, Шийдэл, Бодит туршилт) хичээлийн хэлбэрт шилжүүлсэн: эхний дөрвөн дэлгэцэд philosopher дээр дарж төлөв шилжүүлэх, "бүгд зүүн fork-оо авбал" алхам алхмаар deadlock үүсгэх (circular-wait сумтай), monitor шийдлийг алхам бүрийн тайлбартай ("Яг одоо" мөр, хоолны тооны bar) харуулах зэргээр асуудлыг ойлгуулдаг; сурах бичгийн алгоритм, dp.c, event формат, server.py-ийн /run хэвээр. Мөн ажиллагаа дууссаны дараа deadlock-ийн зураг fork-уудаа чөлөөтэй үзүүлж байсан (cleanup event) алдааг model.js-д засаж, unit тест нэмэв (26 → 30 passed).

Хавсралт Д — Хоёрдугаар шалгалтын (@myk, 2026-10-27) хариу

Нэгдүгээр шүүмжийн засваруудыг хөтөч дээр бодитоор ажиллуулж, нарийвчилсан дахин шалгалт хийсэн хоёр дахь шүүмж ирснийг энд тэмдэглэв. "Заавал засах" (А) ангиллын 1–7 зүйлд хариулав; "Сайжруулах" (Б) ангилал зөвхөн UI-ийн хэмжээний санал (hero хэмжээ, hover дохио, fork-ийн дүрс гэх мэт ~10 санал) бөгөөд хариу өгөх үед хэрэгжээгүй байсан; дараа нь хэсэгчлэн хэрэгжүүлсэн (доорх тэмдэглэлийг үз).

Хүснэгт Д.1 — Хоёрдугаар шалгалтын "Заавал засах" (А) зүйлсийн хариу

#	Асуудал	Хариу
A1	Manual-ийн `cd -FCSA_302_...` команд bash дээр "invalid option" алдаатай (undashed-first-char).	ЗАССАН. `cd ./-FCSA_302_II-4-11-dining-philosophers` болгож Хэрэглэгчийн БА Operational заавар хоёуланд нь засав; `cd ./ бодитоор ажиллуулж шалгасан.
A2	Тестийн тоо хэд хэдэн газар хуучин (16) хэвээр үлдсэн.	ЗАССАН. README, Report (Тойм, 8-р хэсэг, 10-р хэсэг, Хавсрал ийн бодит stdout), Manual — бүгд 19/19 болгосон; README-и жагсаалтад Stop-ийн 2 шинэ шалгалтыг нэмсэн.
A3	README-д Stop-ийн засвар дурдаагүй; Manual "~1 секундийн дотор" гэж хэт нарийвчилсан.	ЗАССАН. README-ийн Known issues-д дэлгэрэнгүй бичлэг нэм Manual-ийн хэллэгийг "хамгийн ихдээ ~2 секунд" болгож, TC холболтын давхар-бичилтийн шалтгааныг тайлбарлав (нэг pi "амжилттай" бичигдээд, ХОЁР дахь дээр л алдаа буцах тохиолдол).
A5	Хавсралт Г-д байгаа нэг лавлагаа тайлангаас олдохгүй; Хавсралт А-ийн commit баганыг git log-оор баталгаажуулаагүй.	ЗАССАН. Лавлагааг хассан (одоо "ХИЙГДЭЭГҮЙ (төлөвлөгөө: хэлбэрээр шууд бичсэн). Хавсралт А-д бодит `git log` гаралты нэмсэн (дээрх хэсгийг үз).
A6	Шинэ describeEvent (fork текст, ping→null)-д unit тест байхгүй.	ЗАССАН. tests/test_model.js-д eating/hungry/thinking/ping дөр тест нэмэв (22 → 26 passed; дараа нь deadlock cleanup-ийн шалгалт нэмж 30 passed). Мөн event лог (#log)-ийн өргөнөөс гарахгүйн тулд текстийг богиносгов: "fork 2, fork 3 авсан" → " 2,3 авсан".
A7	Fork-ийн хөдөлгөөн smooth бус — CSS transition	ЗАССАН (язгуур шалтгаанаар). Fork бүрийг `<g class="forkg"

#	Асуудал	Хариу
	x1/y1/x2/y2 дээр бичсэн ч <line>-ийн эдгээр нь CSS-ээр animate хийгддэггүй тул Chromium дээр 82 хөдөлгөөний БҮГД (82/82) 20px-ээс том шууд үсрэлт үзүүлсэн (дундаж алхам 77.5px) — бодит хэмжилтээр нотлогдсон.	transform="translate(..) rotate(..)">`-д шилжүүлж, `transform`-(SVG-ийн цорын ганц бүх орчинд CSS-ээр найдвартай animat хийгддэг геометрийн шинж чанар) л CSS transition-д оруулав ийн local координат (x1/y1/x2/y2) байрлалаас үл хамаарч тог болгов. Дахин хэмжилт хийхэд sample-ийн зай 0→аажим өсөлт→оргил→аажим буурал→0 хэлбэртэй (жишээ: 0,0, …,2.3,10.8,15.4,13.1,5.0,…,0.9,0) буюу жинхэнэ ease-curve ажиглагдав — өмнөх "нэг л алхамд 77.5px" instant үсрэлт ажиглагдахгүй болсон.

Хоёр дахь шүүмжийн "Б. Сайжруулах санаанууд" (hero-г жижигрүүлэх, тохиргоог нэг мөр болгох, fork-ыг хоёр шүдтэй дүрс болгох, hungry/eating/thinking дохио, deadlock-ийн circular-wait сум, meals bar chart, auto-play/хурд, мобайл CSS order, Host/Origin header шалгалт зэрэг ~10 санал) анх хэрэгжээгүй байсан. Дараа нь дараах нь хэрэгжсэн: Host/Origin шалгалт (server.py, автомат тест нэмсэн); hero-г багасгаж UI-г 5 алхамтай болгосон; хичээлийн дэлгэцүүдэд хоёр шүдтэй fork дүрс, thinking/hungry/eating дохио, deadlock-ийн circular-wait сум, хоолны тооны bar, түр зогсоох/1 алхам удирдлага, мобайл layout. Бодит туршилтын дэлгэц (5) өмнөх хэвээрээ, auto-play хурдны сонголт цаашдын ажил. "В. Өөрчлөхгүй зүйлс" (monitor/naive алгоритм, event формат, HTML id-ууд, гадаад library ашигладаггүй байдал) бүгд хэвээрээ хадгалагдсан.